

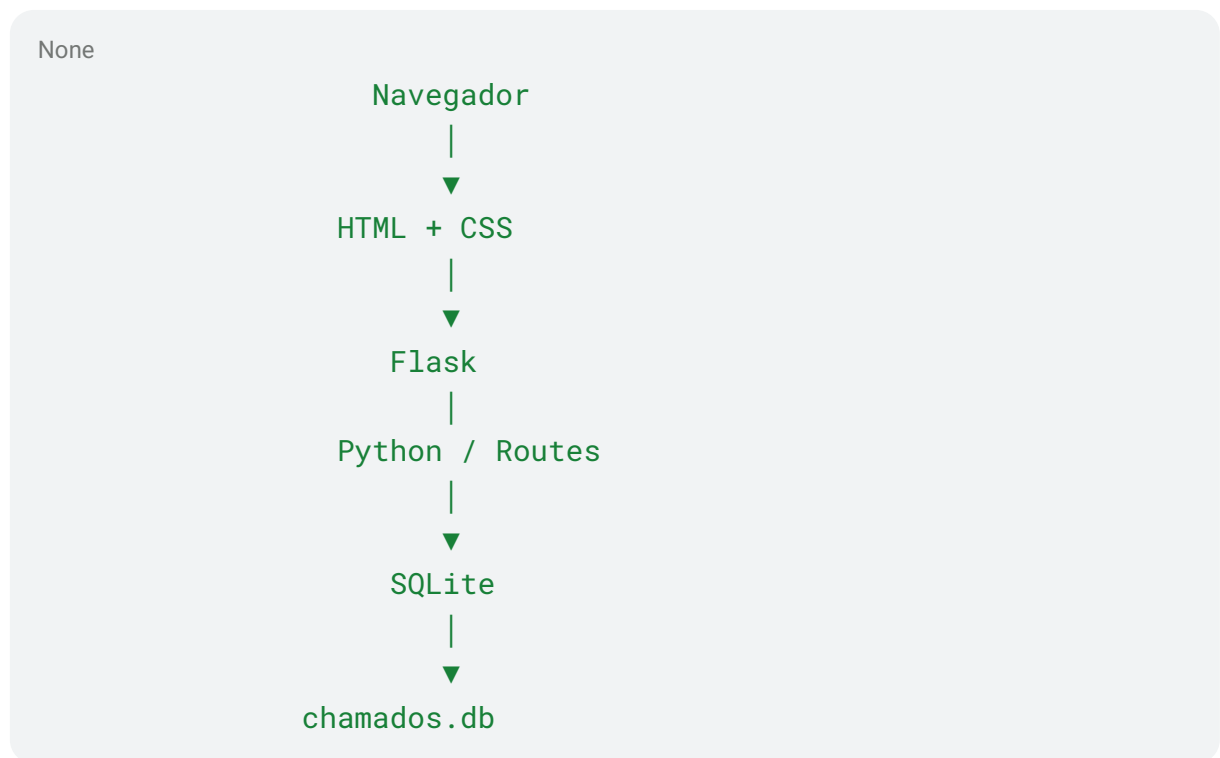
Aula Final — Projeto colaborativo com Git, GitHub e Kanban

Projeto: HelpDesk — Sistema de Chamados

Stack: Python + Flask + SQLite + HTML + CSS + Git + GitHub

Objetivo principal: o sistema é o meio para praticar **Agile, Kanban, Issues, Branches, Commits, Pull Requests, Code Review e Merge.**

Ao final teremos:



1. O sistema que vamos construir

O novo sistema terá:

- Dashboard;
- abertura de chamado;
- listagem de chamados;
- visualização individual;
- alteração de status;
- exclusão de chamado;

- prioridade;
- solicitante;
- data de criação;
- banco SQLite;
- interface web.

Os status serão:

- ABERTO
- EM ATENDIMENTO

FECHADO

E prioridades:

- BAIXA
- MÉDIA

ALTA

2. Estrutura final

Vamos organizar melhor o projeto:

```
None
sistema-chamados/
|
├── app.py
├── database.py
├── chamado.py
|
├── instance/
│   └── chamados.db
|
├── templates/
│   ├── base.html
│   ├── index.html
│   ├── chamados.html
│   ├── novo_chamado.html
│   └── chamado.html
|
└── static/
```

```
|   └─ style.css  
|  
├─ requirements.txt  
├─ .gitignore  
└─ README.md
```

3. Criando o projeto

Crie a pasta:

```
Shell  
mkdir sistema-chamados  
cd sistema-chamados
```

Inicialize o Git:

```
Shell  
git init
```

Crie o ambiente virtual:

```
Shell  
python3 -m venv .venv
```

No macOS/Linux:

```
Shell  
source .venv/bin/activate
```

Instale o Flask:

```
Shell  
pip install flask
```

Crie o `requirements.txt`:

Shell

```
pip freeze > requirements.txt
```

4. `.gitignore`

Crie:

None

```
.venv/  
__pycache__/  
*.pyc  
instance/*.db  
.DS_Store
```

Aqui há uma discussão importante para a aula:

Por que não devemos mandar `.venv` para o GitHub?

Porque outro desenvolvedor pode recriar as dependências com:

Shell

```
pip install -r requirements.txt
```

E por que não enviar o banco?

Porque `chamados.db` contém dados locais e muda constantemente.

5. Banco de dados — `database.py`

Usaremos o `sqlite3`, que já faz parte do Python.

Python

```
import sqlite3
```

```

import os

DATABASE = "instance/chamados.db"

def conectar():
    os.makedirs("instance", exist_ok=True)
    conexao = sqlite3.connect(DATABASE)
    conexao.row_factory = sqlite3.Row
    return conexao

def criar_banco():
    conexao = conectar()
    conexao.execute("""
        CREATE TABLE IF NOT EXISTS chamados (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            solicitante TEXT NOT NULL,
            titulo TEXT NOT NULL,
            descricao TEXT NOT NULL,
            prioridade TEXT NOT NULL,
            status TEXT NOT NULL DEFAULT 'ABERTO',
            criado_em TIMESTAMP DEFAULT CURRENT_TIMESTAMP
        )
    """)
    conexao.commit()
    conexao.close()

```

Explique:

```

None
Flask
  ↓
sqlite3
  ↓
chamados.db
  ↓
Tabela chamados

```

Não precisamos instalar MySQL, PostgreSQL ou Docker.

6. Classe Chamado

chamado.py:

Python

```
class Chamado:
    def __init__(
        self,
        id,
        solicitante,
        titulo,
        descricao,
        prioridade,
        status,
        criado_em=None
    ):
        self.id = id
        self.solicitante = solicitante
        self.titulo = titulo
        self.descricao = descricao
        self.prioridade = prioridade
        self.status = status
        self.criado_em = criado_em
```

Aqui você ainda consegue fazer uma pequena conexão com POO.

7. Aplicação Flask

app.py:

Python

```
from flask import Flask, render_template, request, redirect, url_for

chamado = conexao.execute(
    "SELECT * FROM chamados WHERE id = ?",
    (id,)
).fetchone()
```

```

        conexao.close()
    if chamado is None:
        return "Chamado não encontrado", 404
    return render_template(
        "chamado.html",
        chamado=chamado
    )

@app.route("/chamados/<int:id>/status", methods=["POST"])
def alterar_status(id):
    status = request.form["status"]
    conexao = conectar()
    conexao.execute(
        """
        UPDATE chamados
        SET status = ?
        WHERE id = ?
        """,
        (status, id)
    )
    conexao.commit()
    conexao.close()
    return redirect(
        url_for("visualizar_chamado", id=id)
    )

@app.route("/chamados/<int:id>/excluir", methods=["POST"])
def excluir_chamado(id):
    conexao = conectar()
    conexao.execute(
        "DELETE FROM chamados WHERE id = ?",
        (id,)
    )
    conexao.commit()
    conexao.close()
    return redirect(url_for("listar_chamados"))

if __name__ == "__main__":
    app.run(debug=True)

```

Um detalhe: usamos `?` nos comandos SQL em vez de concatenar texto recebido do formulário. Além de ser mais correto, isso ajuda a evitar SQL Injection.

8. Template principal

templates/base.html:

HTML

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
  <meta charset="UTF-8">
  <title>HelpDesk</title>
  <link
    rel="stylesheet"
    href="{{ url_for('static', filename='style.css') }}"
  >
</head>
<body>
<header>
  <div class="container">
    <h1>HelpDesk</h1>
    <nav>
      <a href="/">Dashboard</a>
      <a href="/chamados">Chamados</a>
      <a href="/chamados/novo">Novo chamado</a>
    </nav>
  </div>
</header>
<main class="container">
  {% block conteudo %}
  {% endblock %}
</main>
</body>
</html>
```

9. Dashboard

templates/index.html:

HTML

```
{% extends "base.html" %}
{% block conteudo %}
<h2>Dashboard</h2>
<div class="dashboard">
  <div class="card">
    <h3>Total</h3>
    <strong>{{ total }}</strong>
  </div>
  <div class="card">
    <h3>Abertos</h3>
    <strong>{{ abertos }}</strong>
  </div>
  <div class="card">
    <h3>Em atendimento</h3>
    <strong>{{ atendimento }}</strong>
  </div>
  <div class="card">
    <h3>Fechados</h3>
    <strong>{{ fechados }}</strong>
  </div>
</div>
{% endblock %}
```

Isso já deixa o projeto com uma aparência muito mais próxima de uma aplicação real.

10. Novo chamado

templates/novo_chamado.html:

HTML

```
{% extends "base.html" %}
{% block conteudo %}
```

```
<h2>Abrir chamado</h2>
<form method="POST">
  <label>Solicitante</label>
  <input
    type="text"
    name="solicitante"
    required
  >
  <label>Título</label>
  <input
    type="text"
    name="titulo"
    required
  >
  <label>Descrição</label>
  <textarea
    name="descricao"
    required
  ></textarea>
  <label>Prioridade</label>
  <select name="prioridade">
    <option value="BAIXA">
      Baixa
    </option>
    <option value="MEDIA">
      Média
    </option>
    <option value="ALTA">
      Alta
    </option>
  </select>
  <button type="submit">
    Abrir chamado
  </button>
</form>
{% endblock %}
```

11. Listagem

templates/chamados.html:

HTML

```
{% extends "base.html" %}
{% block conteudo %}
<div class="titulo-pagina">
  <h2>Chamados</h2>
  <a
    class="botao"
    href="/chamados/novo"
  >
    + Novo chamado
  </a>
</div>
<table>
  <thead>
    <tr>
      <th>ID</th>
      <th>Título</th>
      <th>Solicitante</th>
      <th>Prioridade</th>
      <th>Status</th>
      <th></th>
    </tr>
  </thead>
  <tbody>
    {% for chamado in chamados %}
    <tr>
      <td>
        #{{ chamado.id }}
      </td>
      <td>
        {{ chamado.titulo }}
      </td>
      <td>
        {{ chamado.solicitante }}
      </td>
      <td>
```

```

                {{ chamado.prioridade }}
            </td>
            <td>
                {{ chamado.status }}
            </td>
            <td>
                <a href="/chamados/{{ chamado.id }}">
                    Visualizar
                </a>
            </td>
        </tr>
    {% else %}
    <tr>
        <td colspan="6">
            Nenhum chamado cadastrado.
        </td>
    </tr>
    {% endfor %}
</tbody>
</table>
{% endblock %}

```

12. Visualização do chamado

templates/chamado.html:

```

HTML
{% extends "base.html" %}
{% block conteudo %}
<h2>
    Chamado #{{ chamado.id }}
</h2>
<div class="detalhes">
    <p>
        <strong>Solicitante:</strong>
        {{ chamado.solicitante }}
    </p>

```

```
<p>
    <strong>Título:</strong>
    {{ chamado.titulo }}
</p>
<p>
    <strong>Descrição:</strong>
    {{ chamado.descricao }}
</p>
<p>
    <strong>Prioridade:</strong>
    {{ chamado.prioridade }}
</p>
<p>
    <strong>Status:</strong>
    {{ chamado.status }}
</p>
<p>
    <strong>Criado em:</strong>
    {{ chamado.criado_em }}
</p>
</div>
<h3>Alterar status</h3>
<form
    method="POST"
    action="/chamados/{{ chamado.id }}/status"
>
    <select name="status">
        <option value="ABERTO">
            Aberto
        </option>
        <option value="EM ATENDIMENTO">
            Em atendimento
        </option>
        <option value="FECHADO">
            Fechado
        </option>
    </select>
    <button type="submit">
        Atualizar
    </button>
```

```
</form>
<hr>
<form
  method="POST"
  action="/chamados/{{ chamado.id }}/excluir"
>
  <button
    class="danger"
    type="submit"
  >
    Excluir chamado
  </button>
</form>
{% endblock %}
```

13. CSS

static/style.css:

```
CSS
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f4f6f8;
  color: #222;
}

.container {
  width: 90%;
  max-width: 1100px;
  margin: auto;
}
```

```
header {
  background: #20232a;
  color: white;
  padding: 20px 0;
}

header .container {
  display: flex;
  justify-content: space-between;
  align-items: center;
}

nav a {
  color: white;
  text-decoration: none;
  margin-left: 20px;
}

main {
  padding-top: 40px;
}

.dashboard {
  display: grid;
  grid-template-columns: repeat(4, 1fr);
  gap: 20px;
}

.card {
  background: white;
  padding: 25px;
  border-radius: 8px;
}

.card strong {
  font-size: 32px;
}

form {
  background: white;
```

```
        padding: 25px;
        border-radius: 8px;
        max-width: 700px;
    }

    label {
        display: block;
        margin-top: 15px;
        margin-bottom: 5px;
    }

    input,
    textarea,
    select {
        width: 100%;
        padding: 10px;
    }

    textarea {
        min-height: 120px;
    }

    button,
    .botao {
        display: inline-block;
        margin-top: 20px;
        padding: 10px 18px;
        border: 0;
        border-radius: 5px;
        background: #20232a;
        color: white;
        text-decoration: none;
        cursor: pointer;
    }

    .danger {
        background: #b42318;
    }

    table {
```

```
        width: 100%;
        background: white;
        border-collapse: collapse;
    }

    th,
    td {
        padding: 15px;
        text-align: left;
        border-bottom: 1px solid #ddd;
    }

    .titulo-pagina {
        display: flex;
        justify-content: space-between;
        align-items: center;
    }

    .detalhes {
        background: white;
        padding: 25px;
        border-radius: 8px;
        margin-bottom: 20px;
    }
```

14. Executando

No terminal:

```
Shell
python app.py
```

O Flask mostrará algo parecido com:

```
None
Running on http://127.0.0.1:5000
```

Abra o endereço no navegador.

Agora temos:

```
None
Dashboard
  |
  ├── Total
  ├── Abertos
  ├── Em atendimento
  └── Fechados

Chamados
  |
  ├── Listar
  ├── Visualizar
  ├── Alterar status
  └── Excluir

Novo chamado
  |
  └── Cadastrar → SQLite
```

15. Agora começa a verdadeira aula de Agile

Crie o Kanban:

```
None
BACKLOG
TO DO
IN PROGRESS
REVIEW
DONE
```

Estamos adicionado **REVIEW** nesta última aula justamente por causa dos Pull Requests.

16. Criando o Product Backlog

Podemos dividir o sistema em Issues:

Issu e	História
#1	Criar estrutura Flask
#2	Criar banco SQLite
#3	Criar abertura de chamados
#4	Criar listagem
#5	Criar visualização
#6	Criar alteração de status
#7	Criar exclusão
#8	Criar Dashboard
#9	Criar layout
#10	Melhorar CSS

Agora podemos introduzir **User Stories**.

Em vez de:

Criar chamado.

Escrevemos:

Como usuário, quero abrir um chamado informando meu problema para que a equipe de suporte possa atendê-lo.

Critérios de aceitação:

- Informar solicitante
- Informar título
- Informar descrição
- Selecionar prioridade
- Status inicial deve ser ABERTO

Chamado deve ser salvo no banco

Isso aproxima muito mais a aula de Agile.

17. Divisão da equipe

Suponha quatro desenvolvedores:

Desenvolvedor A

Issue #3

Abertura de chamados

Desenvolvedor B

Issue #4

Listagem de chamados

Desenvolvedor C

Issue #6

Alteração de status

Desenvolvedor D

Issue #8

Dashboard

Cada um move:

None

TO DO



IN PROGRESS



18. Atualizando antes de trabalhar

Todo desenvolvedor começa:

```
Shell  
git switch main
```

Depois:

```
Shell  
git pull origin main
```

E somente então cria sua branch.

Desenvolvedor A:

```
Shell  
git switch -c feature/abrir-chamado
```

Desenvolvedor B:

```
Shell  
git switch -c feature/listar-chamados
```

Desenvolvedor C:

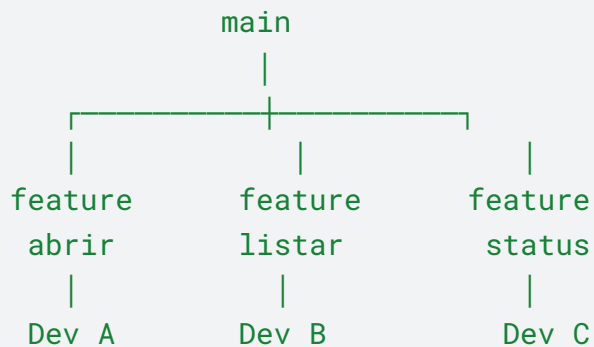
```
Shell  
git switch -c feature/status-chamado
```

Desenvolvedor D:

```
Shell  
git switch -c feature/dashboard
```

Temos:

None



19. Desenvolveu? Commit

Exemplo:

Shell

```
git status
git add .
git commit -m "feat: adiciona abertura de chamados"
```

E:

Shell

```
git push -u origin feature/abrir-chamado
```

Fazendo o Merge

Existem **duas formas principais de realizar o merge via terminal**, e comparar essa diferença é uma excelente oportunidade didática para a aula.

1. Merge do Pull Request utilizando o GitHub CLI

Se você está seguindo o fluxo de trabalho com Pull Request, esta é a opção mais recomendada.

Primeiramente, liste os Pull Requests abertos:

```
Shell  
gh pr list
```

Saída esperada no terminal:

```
None  
#3 feat: implementa abertura de chamados
```

Para visualizar os detalhes de um PR específico:

```
Shell  
gh pr view 3
```

Para executar o merge do PR:

```
Shell  
gh pr merge 3
```

O GitHub CLI solicitará a escolha da estratégia de merge. Você também pode especificá-la diretamente via flag:

```
Shell  
gh pr merge 3 --merge
```

Ou para consolidar os commits (squash):

```
Shell  
gh pr merge 3 --squash
```

Ou aplicando rebase:

```
Shell  
gh pr merge 3 --rebase
```

Para fins didáticos, recomenda-se iniciar com o merge tradicional:

Shell

```
gh pr merge 3 --merge
```

Em seguida, atualize o repositório local na branch `main`:

Shell

```
git switch main
```

Shell

```
git pull origin main
```

O fluxo completo de integração é estruturado da seguinte forma:

None

```
feature/abrir-chamado
  |
  | git push
  ▼
GitHub
  |
  | gh pr create
  ▼
Pull Request
  |
  | gh pr merge
  ▼
origin/main
  |
  | git pull
  ▼
main local
```

2. Merge direto utilizando o Git

Também é possível realizar o merge diretamente sem criar um Pull Request:

Shell

```
git switch main
```

Shell

```
git merge feature/abrir-chamado
```

E envie as alterações para o repositório remoto:

Shell

```
git push origin main
```

Nesta abordagem, a estrutura segue este caminho:

None

```
feature/abrir-chamado
```

↓

```
git merge
```

↓

```
main local
```

↓

```
git push
```

↓

```
origin/main
```

Contudo, essa estratégia não é recomendada como fluxo principal, dado que o objetivo principal é exercitar o fluxo colaborativo no GitHub, o uso de Pull Requests e o acompanhamento via Kanban, garantindo a revisão de código antes da integração.

Portanto, o fluxo de trabalho ideal a ser apresentado é o seguinte:

```
Shell
# O desenvolvedor cria a branch para a feature
git switch -c feature/abrir-chamado

# Desenvolve as alterações necessárias...

# Adiciona, realiza o commit e envia a branch ao remoto
git add .
git commit -m "feat: implementa abertura de chamados"
git push -u origin feature/abrir-chamado

# Cria o Pull Request via CLI
gh pr create

# Outro membro do time realiza a revisão e aprova
gh pr review 3 --approve

# O responsável faz o merge do PR
gh pr merge 3 --merge

# Todos os membros atualizam a branch main local
git switch main
git pull origin main
```

Esse modelo simula fielmente o fluxo de desenvolvimento em equipe utilizado no mercado de trabalho: **desenvolvedor** → **PR** → **revisão** → **aprovação** → **merge** → **atualização da main**.

20. Pull Request sem entrar no GitHub

Agora usamos o que vocês já aprenderam:

```
Shell
gh pr create \
  --base main \
  --head feature/abrir-chamado \
  --title "feat: adiciona abertura de chamados" \
  --body "Implementa abertura de chamados. Closes #3"
```

No Kanban:

```
None
IN PROGRESS
  ↓
REVIEW
```

21. Outro Dev faz Code Review

Primeiro:

```
Shell
gh pr list
```

Depois:

```
Shell
gh pr view 3
```

Para visualizar as alterações:

```
Shell
gh pr diff 3
```

Isso é excelente para mostrar em projetor.

Depois o aluno pode aprovar:

```
Shell
gh pr review 3 --approve
```

Agora temos:

```
None
Aluno A
|
```



22. Merge

Depois da aprovação:

Shell
`gh pr merge 3 --merge`

No Kanban:

None
REVIEW
↓
DONE

23. O problema interessante da aula

Agora provocamos uma situação proposital.

O desenvolvedor B continua trabalhando em:

feature/listar-chamados

Mas o desenvolvedor A acabou de colocar código novo na `main`.

Portanto:



A branch dele ficou para trás.

Ele pode atualizar:

Shell

```
git switch main
git pull origin main
```

Depois:

Shell

```
git switch feature/listar-chamados
```

E:

Shell

```
git merge main
```

Agora o começamos a entender **por que conflitos acontecem**.

24. Demonstração de Merge Conflict

Vamos provocar um conflito.

Dois desenvolvedores alteram a mesma região de `base.html`.

Um coloca:

```
HTML
<h1>HelpDesk</h1>
```

Outro:

```
HTML
<h1>Sistema de Suporte</h1>
```

Ao tentar merge, Git pode apresentar:

```
None
CONFLICT (content): Merge conflict in templates/base.html
```

O arquivo poderá mostrar:

```
HTML
<<<<<< HEAD
<h1>HelpDesk</h1>
=====
<h1>Sistema de Suporte</h1>
>>>>>> feature/layout
```

Explicação:

O Git sabe que duas pessoas alteraram o mesmo lugar, mas não sabe qual alteração representa a decisão correta do time.

A equipe decide, por exemplo:

```
HTML
<h1>HelpDesk - Sistema de Suporte</h1>
```

Depois:

Shell

```
git add templates/base.html  
git commit -m "fix: resolve conflito no titulo do sistema"
```

E:

Shell

```
git push
```

Isso fecha muito bem a parte prática de Git.

26. Retrospectiva — fechando Agile

Retrospectiva.

None

RETROSPECTIVA



0 que funcionou?



0 que não funcionou?



0 que podemos melhorar?

Pergunta:

O que funcionou?

Exemplo:

Dividir funcionalidades entre membros.

O que não funcionou?

Duas pessoas editaram o mesmo arquivo e tivemos conflito.

O que podemos melhorar?

Comunicar melhor quais partes cada pessoa está alterando.

E agora conseguimos fechar o ciclo:

None



Assim, a última aula deixa de ser simplesmente **"mais uma aula sobre comandos Git"**. Ela amarra as disciplinas do módulo: **Git + GitHub + Agile + Kanban + colaboração + Dev Culture**, usando um pequeno software real como laboratório.